CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 10/Unit 10: Inheritance-1

Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk
)

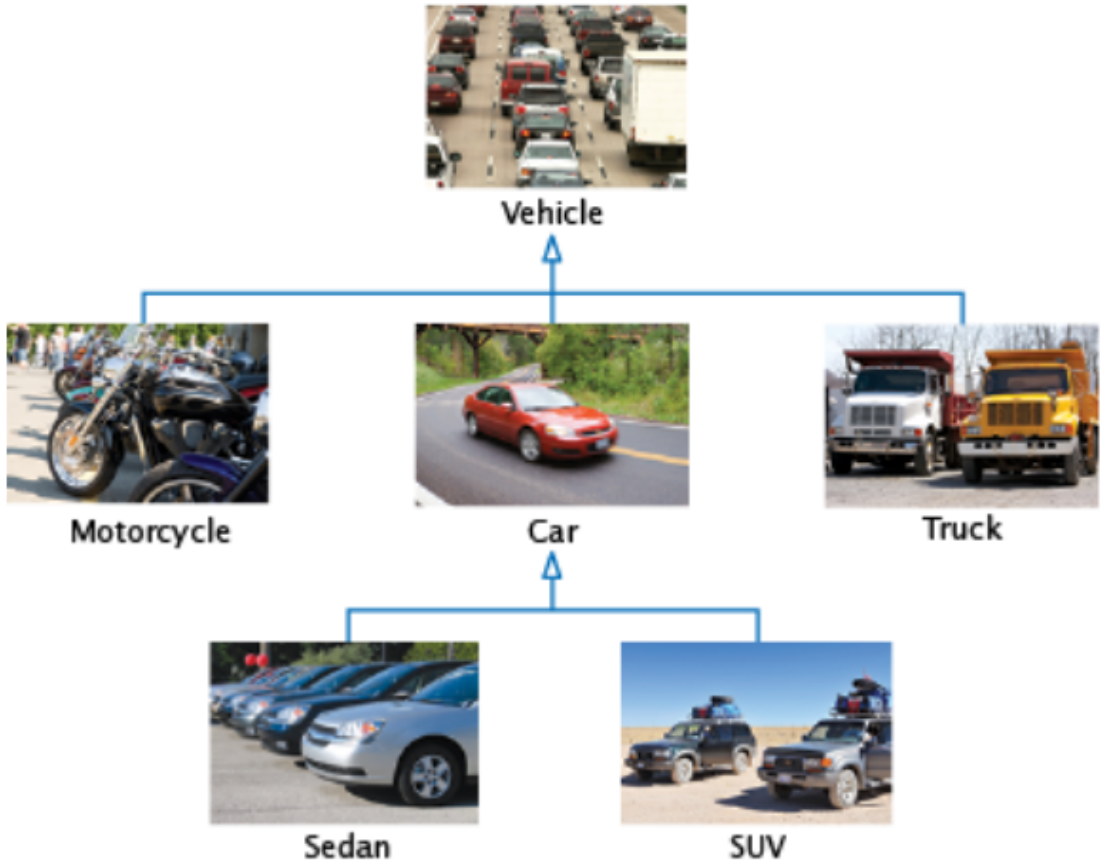
Unit Outline

1. Introduction
2. Inheritance
3. Inheritance Hierarchy
4. Question Class
5. Derived Classes
6. Exercises

Inheritance

- In object-oriented design, **inheritance** *is a* relationship between a more general class (called the **base class**) and a more specialized class (called the **derived class**).
- The terms (superclass, subclass) and (parent class, child class) are also used.
- The derived class inherits data and behavior from the base class.
- Inheritance is basically and “is-a” relationship between a general base class and a specialized derived class.

An inheritance hierarchy

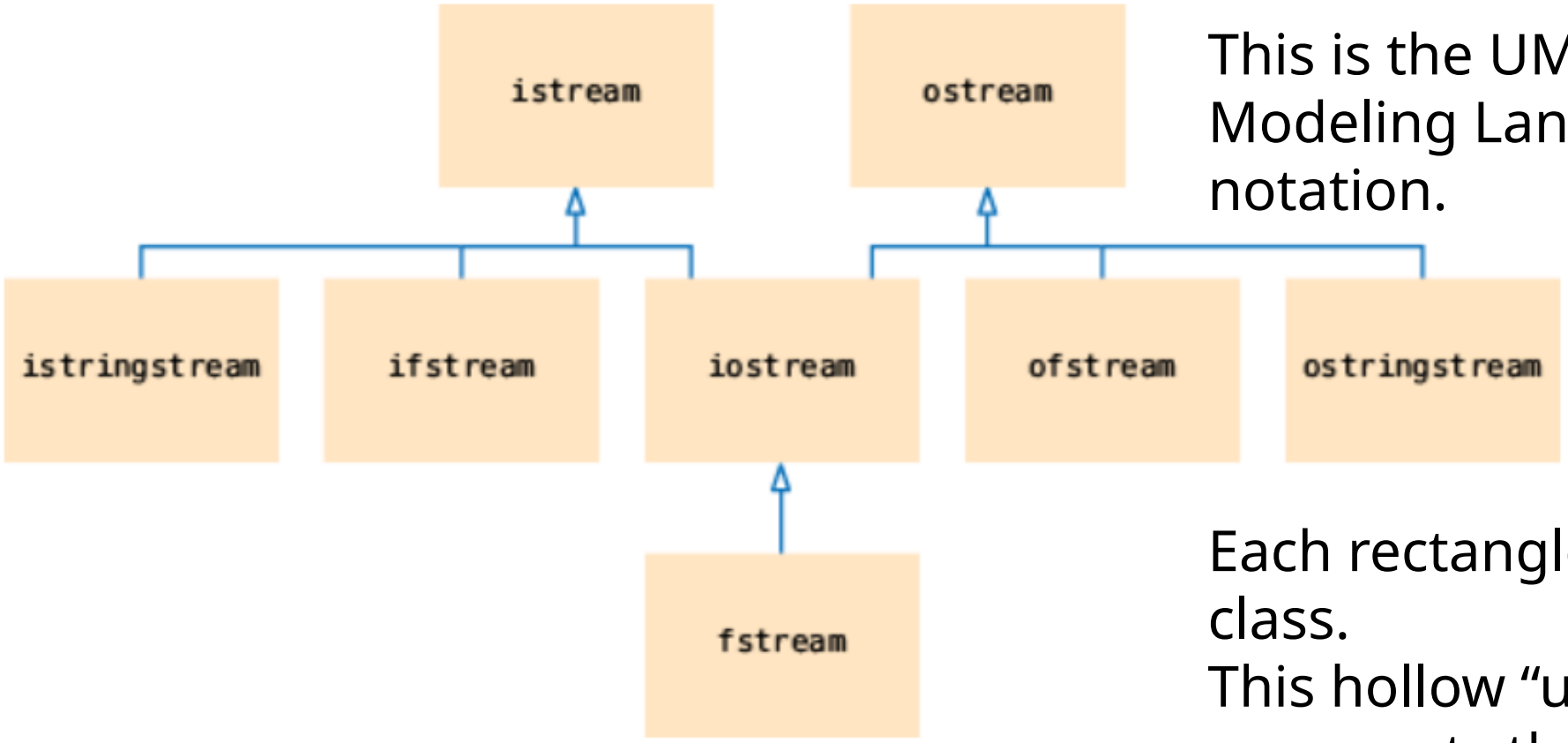


- Cars share common traits of vehicles, such as the ability to transport people.
- The class Car inherits from the class Vehicle.
- The Vehicle class is the base class and the Car class is the derived class.

Inheritance

- The inheritance relationship is very powerful because it allows us to reuse algorithms with objects of different classes.
- Suppose we have an algorithm that manipulates a Vehicle object.
- Because a *car* is a special kind of *vehicle*, we can supply a Car object to such an algorithm, and it will work correctly.
- This is an example of the **substitution principle** that states that you can always use a derived-class object when a base-class object is expected.

Inheritance hierarchy of C++ IO Classes



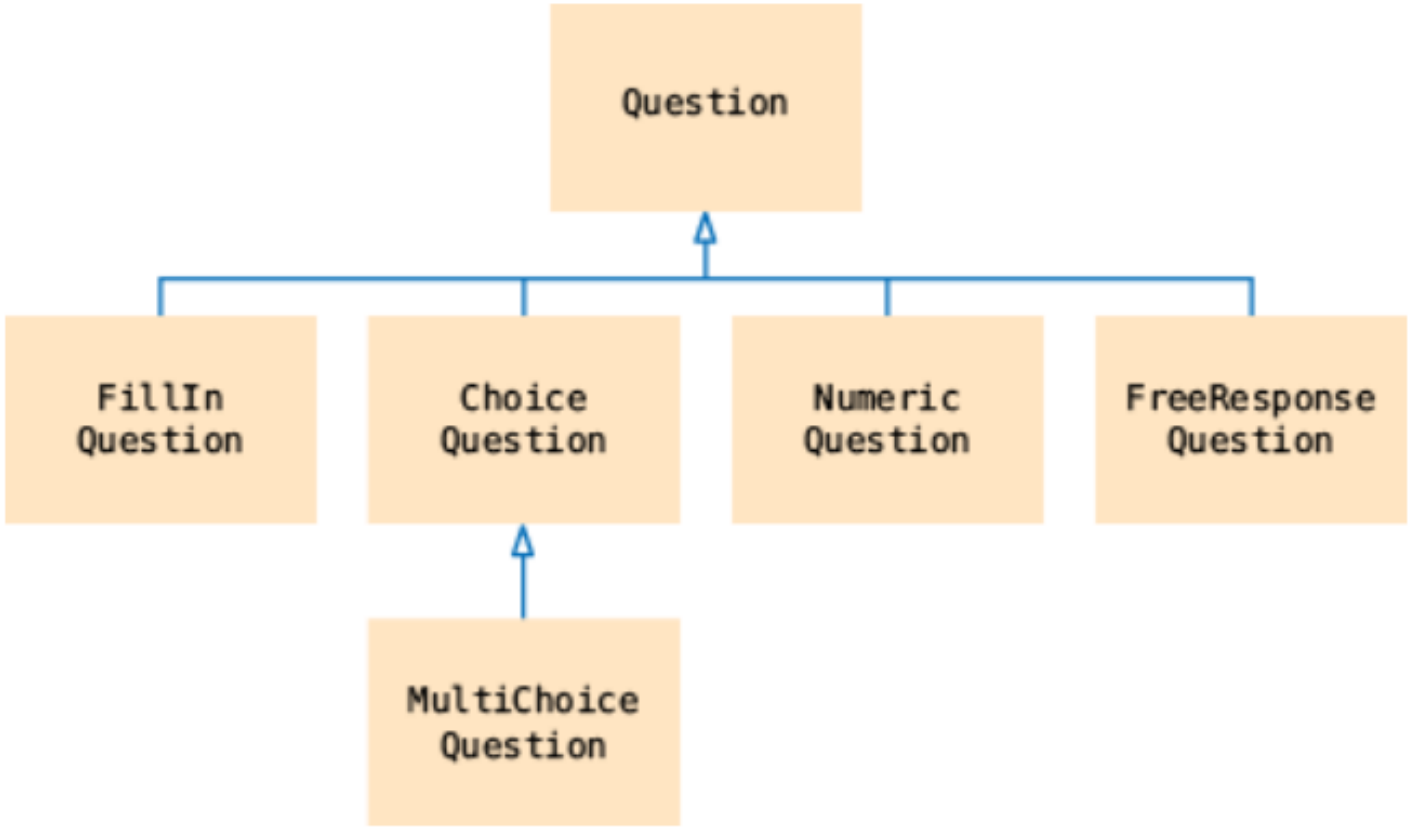
This is the UML (Unified Modeling Language) notation.

Each rectangle represents a class.
This hollow “up” arrow represents the inheritance relationship.

Questions Hierarchy - Example

- Consider a simple hierarchy of questions in a quiz. A quiz consists of different kinds of questions such as:
  - Fill-in-the-blank
  - Choice (single or multiple)
  - Numeric (where an approximate answer is ok; e.g., 1.33 when the actual answer is 4/3)
  - Free response
- An inheritance hierarchy for these question types can be constructed.

Questions Hierarchy - Example



The Question class

- At the root of this hierarchy is the Question type.
- A question can display its text, and it can check whether a given response is a correct answer.

```
class Question
{
public:
   Question();
   void set_text(string question_text);
   void set_answer(string
correct_response);
   bool check_answer(string response)
const;
   void display() const;
private:
   string text;
   string answer;
};
```

Self-Test

1) Suppose you are to design an inheritance hierarchy with the following classes: `Snake`, `Crocodile`, `Reptile`, and `Turtle`. Which of these is the base class?

   - ○ `Snake`
   - ○ `Crocodile`
   - ○ `Reptile`
   - ○ `Turtle`

Self-Test

Suppose you have designed an inheritance hierarchy that includes the following relationships:

`Guitar` is derived from `Instrument`
`AcousticGuitar` is derived from `Guitar`
`ElectricGuitar` is derived from `Guitar`

1) Given the declarations below, which of the objects CANNOT be passed to the function `tune(Guitar& g)`?

- ○ `AcousticGuitar ag;`
- ○ `ElectricGuitar eg;`
- ○ `Guitar my_guitar;`
- ○ `Instrument my_instrument;`

Derived Classes

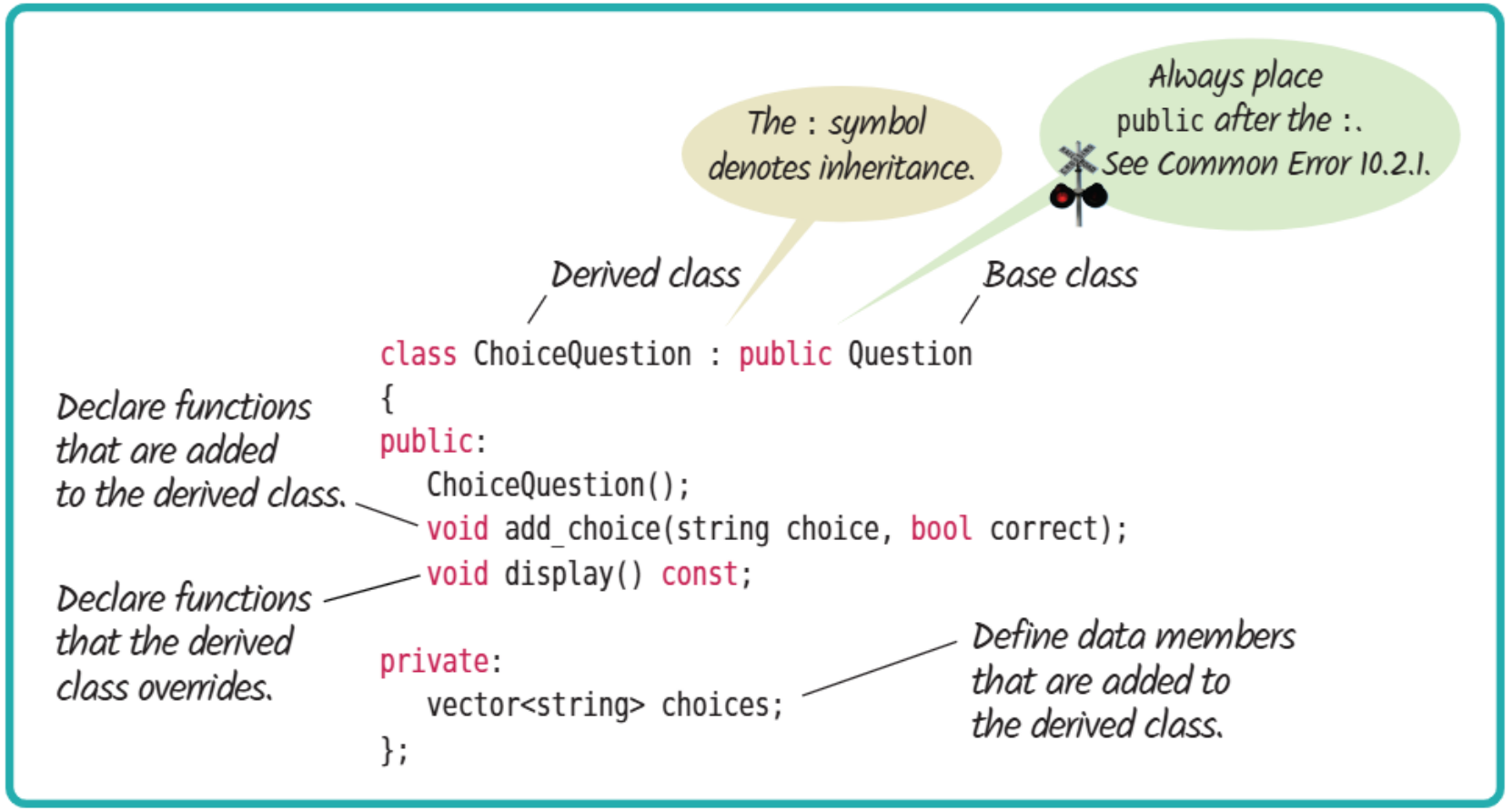
- In C++, you form a derived class from a base class by specifying what makes the derived class different.
- You define the member functions that are new to the derived class.
- The derived class inherits all member functions from the base class,
- You can change the implementation if the inherited behavior is not appropriate.
- The derived class automatically inherits all data members from the base class.
- You only define the added data members.

Derived Classes – ChoiceQuestion

- Here is the syntax for the definition of a derived class:

```
class ChoiceQuestion : public Question
{
public:
   New and changed member functions
private:
   Additional data members
};
```

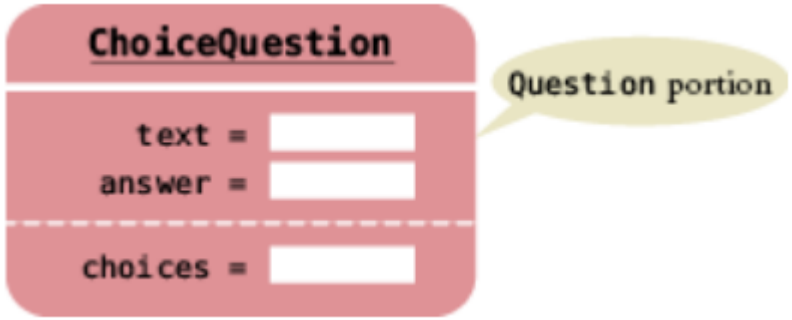
Derived Classes – Format



Derived Class - ChoiceQuestion

- A ChoiceQuestion object differs from a Question object in three ways:
- Its objects store the various choices for the answer.
- There is a member function for adding another choice.
- The display function of the ChoiceQuestion class shows these choices so that the respondent can choose one of them.

```
class ChoiceQuestion :
                public Question
{
public:
   ChoiceQuestion();
   void add_choice(string choice,
                        bool correct);
   void display() const;
private:
   vector<string> choices;
};
```



Derived Class - ChoiceQuestion

- The `add_choice` function is specific to the `ChoiceQuestion` class.
- You can only apply it to `ChoiceQuestion` objects, not general `Question` objects.
- However, the display function is a redefinition of a function that exists in the base class; it is redefined to take into account the special needs of the derived class.
- We say that the derived class **overrides** this function.
- (Definition of **override**: Redefine a function from a base class in a derived class.)

Derived Class - ChoiceQuestion

- The add_choice function

```
void ChoiceQuestion::add_choice(string choice, bool
correct)
{
   choices.push_back(choice); //choices is a vector
   if (correct) {
      int choice_index = choices.size();
      string choice_string = to_string(choice_index);
      set_answer(choice_string);
   }
}
```

Derived Class - ChoiceQuestion

- Overridden display() function

```
void ChoiceQuestion::display() const {
    Question::display();
    cout << "Choices>> ";
    for (const std::string& choice : choices) {
         std::cout << choice << " ";
    }
    std::cout << std::endl;
}
```

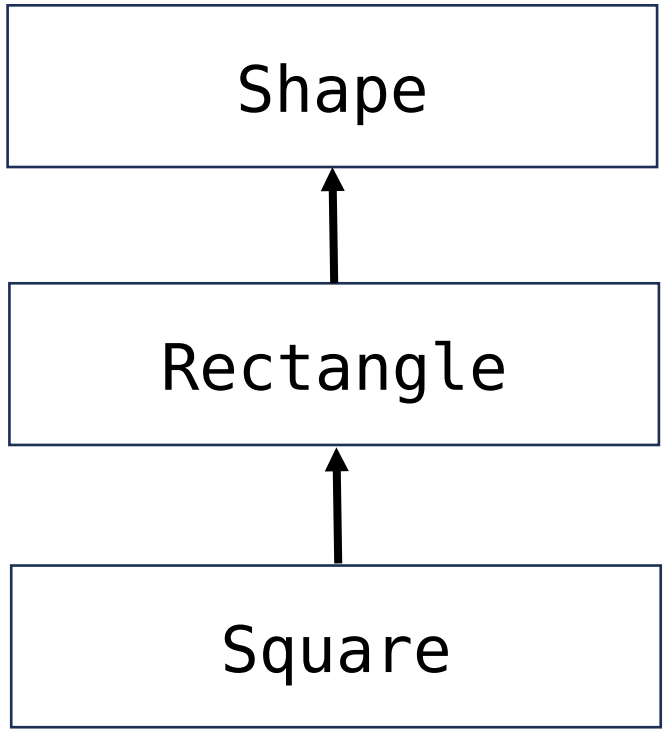
Derived Class - NumericQuestion

```
class NumericQuestion : public Question
{
public:
   void display() const;
   bool check_answer(string response) const;
   void set_answer(double correct_response);
private:
   double expected;
};
void NumericQuestion::display() const
{
   Question::display();
   cout << "Enter a number." << endl;
}
```

```
bool NumericQuestion::check_answer(string response) const
{
   double value = stod(response);
   //return true if value and
   //expected are very close
   return abs(value - expected) < 1E-12;
}
void NumericQuestion::set_answer(double correct_response)
{
   answer = correct_response;
}
```

Another Example

- Consider the following inheritance hierarchy:



The Shape Example – Object Creation Order

```
class Shape {
private:
    string name;
public:
    Shape() {
        cout << “Creating shape" << endl;
    }
};
class Rectangle : public Shape {
private:
    int length;
    int width;
public:
    Rectangle() {
        cout << “Creating rectangle" << endl
    }
};
```

```
class Square : public Rectangle {
public:
    Square() {
        cout << "Creating square" <<
endl;
    }
};

int main() {
    Square square;
    return 0;
}
```

Didn’t we initialize Square only in main? So why are we seeing this output?

```
Creating shape
Creating rectangle
Creating square
```

Object Creation Order

- The previous example shows that in order to create an object, the constructor of its parent (base) class is called.
- This chain continues until we reach the base class.
- In effect, in order to create an object of type Square, we create an object of type Rectangle, which creates an object of type Shape.
- What happens in a constructor of a base class is not present?
- In that case, the default (no-arguments) constructor is called to construct the object.

Object Creation Order

```
class Shape {
private:
    string name;

public:
    Shape(string name) {
        this->name = name;
        cout << "Creating shape" <<
endl;
    }
};
```

```
class Rectangle : public Shape {
private:
    int length;
    int width;

public:
    Rectangle(string name, int length, int
width) {
        this->name = name; //Can I do this?
        this->length = length;
        this->width = width;
        cout << "Creating rectangle" <<
endl;
    }
};
```

Question: We want to create a rectangle, and we want to assign it a name, i.e. initialize its name attribute from its constructor. Can we do this directly as shown here?

Object Creation Order

- The correct way to initialize a parent(super/base) class's variables is to call its constructor, since the variables of the parent(super/base) class are private.
- This can be done using an initializer list as follows:

```
class Shape {
private:
    string name;

public:
    Shape(string n) : name(n) {
        cout << "Now creating shape with name: " << name <<
endl;
    }
};
```

Object Creation Order (contd.)

```
class Rectangle : public Shape {
private:
    int length;
    int width;

public:
    Rectangle(string n, int l, int w) : Shape(n), length(l),
width(w) {
        cout << "Now creating rectangle with length " << length <<
" and width " << width << endl;
    }
};
```

Object Creation Order (contd.)

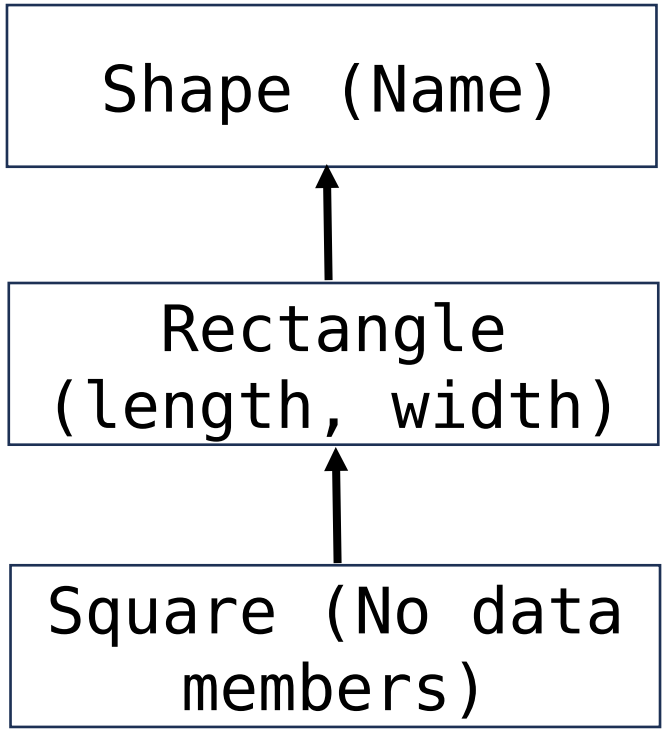
```
class Square : public Rectangle {
public:
    Square(string n, int side) : Rectangle(n, side, side) {
        cout << "Now creating square with side " << side << endl;
    }
};

int main() {
    Square square("MySquare", 5);
    return 0;
}
```

```
Now creating shape with name: MySquare
Now creating rectangle with length 5 and width 5
Now creating square with side 5
```

Object Creation Order – rules

- In C++, you cannot directly call the constructor of the superclass (base class) from within the body of the constructor of the subclass (derived class).
- Instead, you must call the base class constructor in the initializer list of the derived class constructor.
- There is a specific order to follow when initializing members in the initializer list of a constructor in C++.
- The base class constructors are called before the derived class's member initializers are executed.
- If the derived class has multiple base classes, the order of initialization will follow the order in which the base classes are declared in the class definition (from left to right).

Object Creation Order – Rules



- In C++, when an object of a derived class is created, the constructors of its base classes are invoked first.
- This ensures that the base class part of the derived class object is fully initialized before any additional initialization is executed.
- The memory layout of a derived class object includes the memory for its base class parts as well.

Object Destruction Order - Reverse

- In C++, the order of object destruction follows the **reverse** of the construction order.
- When an object is destroyed, the destructors are called in the following order:
- The destructor of the derived class is called first. This allows the derived class to perform any cleanup that is specific to itself.
- After the derived class destructor has completed its execution, the destructor of the base class is called.
- This ensures that any resources or states managed by the base class are properly cleaned up.

Order of Destructors – Program

```
class Base {
public:
    Base()  { std::cout << "Base constructor called." << std::endl; }
    ~Base() { std::cout << "Base destructor called." << std::endl;  }
};
class Derived : public Base {
public:
    Derived()  { std::cout << "Derived constructor called." << std::endl; }
    ~Derived() { std::cout << "Derived destructor called." << std::endl;  }
};
int main() {
    Derived d; // Creating an object of Derived cla
    return 0;
}
```

```
Base constructor called.
Derived constructor called.
Derived destructor called.
Base destructor called.
```

Exercises

- We saw the Shape-Rectangle-Square hierarchy.
- Add a function called int getArea() to this hierarchy? Which class should it be in?
- Should the square class override its getArea() over the Rectangle's getArea()?
- Should the Shape class have a getArea() function? If so, what should it return?
- -Review Exercises from Horstmann, Chapter 10.